

Integrating **Lunar** with a multi-tenant TALL stack approach is a sophisticated move. Since Lunar is "headless" by nature, it provides the backend infrastructure while leaving the frontend (the "Virtual Storefront") entirely to you.

Using the customer-subdomain.top5pct.com approach means your primary task is **context switching**—ensuring that when a user hits a subdomain, Lunar and your Blade components know exactly which "Store" data to fetch.

1. Database & Model Structure

You'll need a Store (or Tenant) model that acts as the source of truth for your specialized Blade components.

The Store Model

PHP

```
// app/Models/Store.php
class Store extends Model
{
    protected $casts = [
        'theme_config' => 'array', // Stores colors, font choice, nav layout
        'features_enabled' => 'array', // Toggle: ['calendar', 'roster', 'countdown']
    ];

    // Link this to Lunar's Channel system
    public function channel()
    {
        return $this->belongsTo(\Lunar\Models\Channel::class);
    }
}
```

The "Current Store" Middleware

This middleware identifies the tenant and "scopes" Lunar to only show products assigned to that specific store's channel.

PHP

```

public function handle($request, Closure $next)
{
    $subdomain = explode('.', $request->getHost())[0];
    $store = Store::where('subdomain', $subdomain)->firstOrFail();

    // Share globally with Blade and Livewire
    app()->instance('current_store', $store);

    // Tell Lunar which channel to use for pricing/visibility
    \Lunar\Facades\Channels::setCurrent($store->channel->handle);

    return $next($request);
}

```

2. Specialized Blade Components

To handle your requirements efficiently, use **Anonymous Blade Components** or **Livewire Components** that read from `app('current_store')`.

Navigation Bar (Dynamic Layout)

Instead of three different files, use one component with conditional CSS classes.

HTML

```

{{-- resources/views/components/nav.blade.php --}}
@php
    $layout = app('current_store')->theme_config['nav_layout']; // 'left', 'right', 'center'
    $flexClass = match($layout) {
        'left' => 'flex-row',
        'right' => 'flex-row-reverse',
        'center' => 'flex-col items-center',
        default => 'flex-row'
    };
@endphp

<nav class="flex {{ $flexClass }}" p-4 bg-brand-primary">

```

```

<div class="links">...</div>
</nav>
```

Tri-Palette CSS Scheme

In your app.blade.php, inject the tenant's specific palette.

HTML

```
<style>
:root {
  --brand-primary: {{ app('current_store')->theme_config['colors']['primary'] }};
  --brand-secondary: {{ app('current_store')->theme_config['colors']['secondary'] }};
  --brand-accent: {{ app('current_store')->theme_config['colors']['accent'] }};
  --main-font: "{{ app('current_store')->theme_config['font_family'] }}" , sans-serif;
}
body { font-family: var(--main-font); }
</style>
```

3. Extending the Lunar Dashboard (The Management Side)

Lunar is built on **Filament**. To manage your Multi-Tenancy inside the Lunar admin, you need to register a new "Resource" in Filament.

Step 1: Create a Store Resource

Run `php artisan make:filament-resource Store`. This adds a "Stores" menu item to your Lunar dashboard.

Step 2: Use "Groups" for UX

Inside your StoreResource.php, use **Tabs** or **Sections** to organize the specialized components you mentioned:

- **Design Tab:** Color pickers for the tri-palette, dropdown for Font choice, and Radio buttons for Nav Layout.
- **Events Tab:** A Repeater field for the "Calendar of Events" (Event Name, Date, Location).
- **Roster Tab:** A file upload for school pictures or a repeater for student names.

4. Do you need a custom Cart and Checkout?

Yes, but not from scratch.

Since you are white-labeling for different sectors (School vs. Business):

1. **Shared Logic:** Use Lunar's underlying Cart and Order models for both.
2. **Specialized UI:** School stores often need a "Homeroom Teacher" or "Student Name" field at checkout. Business stores often need a "Purchase Order Number" field.
3. **The Approach:** Create one base Livewire Checkout component. Use if/else or dynamic sub-views based on the `app('current_store')->type`.

5. Additional Considerations ("Other?")

Since you asked what else you might need:

- **Bulk Image Processing:** For school stores, you'll likely need a way to map photos to products. Look into **Spatie Media Library** (which Lunar uses) to handle the "Roster for pictures."
- **Custom Domain SSL:** If a client eventually wants `shop.schoolname.com` instead of your subdomain, you will need **Laravel Forge's API** to programmatically add domains and request Let's Encrypt certificates.
- **Email White-labeling:** Don't forget that "Order Confirmed" emails should also use the store's logo and colors. Create a base Markdown Mailables template that pulls from `app('current_store')`.

Would you like a code snippet for the "Calendar of Events" repeater logic in the Filament/Lunar dashboard?